

Multi-Agent Systems

Instructor: Your Name

{ik} INTERVIEW KICKSTART



Julie Wilkins

- Senior Product Manager, Amazon
- MBA – Temple University
- BA – Louisiana State University

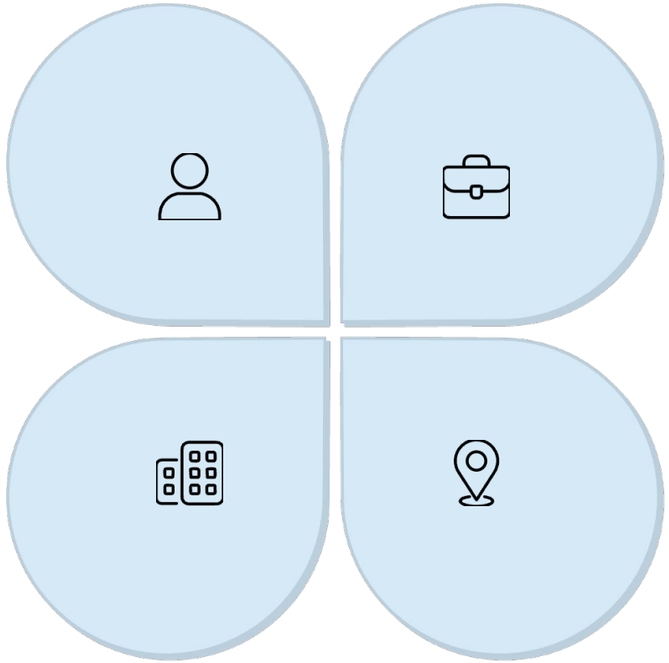
{ik} INTERVIEW KICKSTART

Welcome to today's class, before we begin,

pop into the chat:

Your Name

Your Role



Your Company

Your Location

{ik} INTERVIEW KICKSTART

Optimize Your Experience

Interact with Instructors

Join Sunday live class to ask questions and get clarifications.

Engage with Assessments

Complete assignments and MCQs. Get feedback during Thursday review sessions.

Utilize Your Resources

Make the most of all available resources to maximize your learning experience.

{ik} INTERVIEW KICKSTART



Are you using all these IK support features?



Uplevel Resources

Contains all the learning resources - videos, MCQs, assignments.



Post-Class Videos

Optional videos for additional reference.



Technical Coaching

Sessions on Wednesday to clarify all your queries.



Discord Community

For peer learning and collaboration.



TA Assistance

TAs help answer queries in the live class.



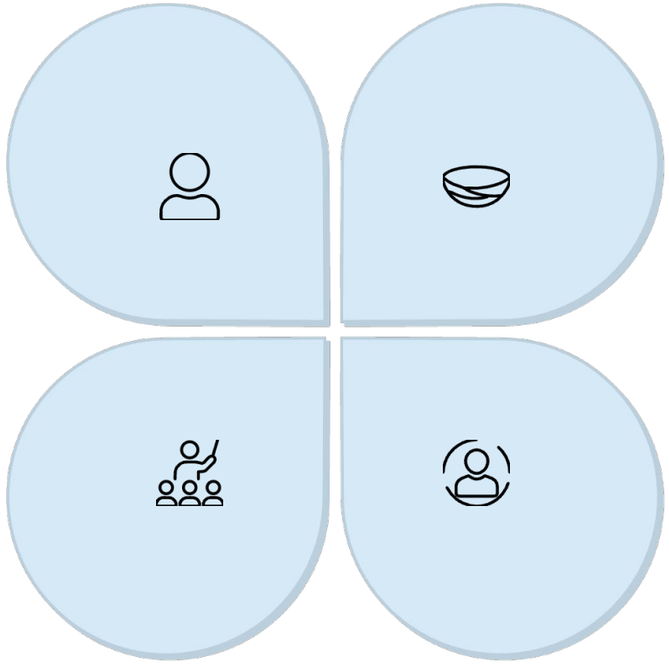
Support Tickets

Raise them any time you want any clarifications/tech concerns, etc on UpLevel.

{ik} INTERVIEW KICKSTART

Learning Objectives

Multi-Agent Systems



Concept of SubAgents

Orchestration/Router

Parallelisation

{ik} INTERVIEW KICKSTART

Before We Begin

We assume that from previous lectures you already have understanding of:

- **LLMs**
- **AI Agents**
- **RAG**
- **Tools & Tool Calls**
- **Familiarity with LangChain**

{ik} INTERVIEW KICKSTART

MICROSOFT + CONTRAFORCE: Cybersecurity Revolution

The Problem:

Managed Service Providers (MSPs) were drowning in cybersecurity incidents. Average response time was **30 minutes per incident** at a cost of **$15 per incident**—unsustainable at scale.

How these were addressed earlier?

- Analyzing security data
- Identifying suspicious activities
- Managing incidents
- Creating investigation reports

Source: https://www.microsoft.com/en-us/microsoft-cloud/blog/2025/12/04/multi-agentic-ai-unlocking-the-next-wave-of-business-transformation/

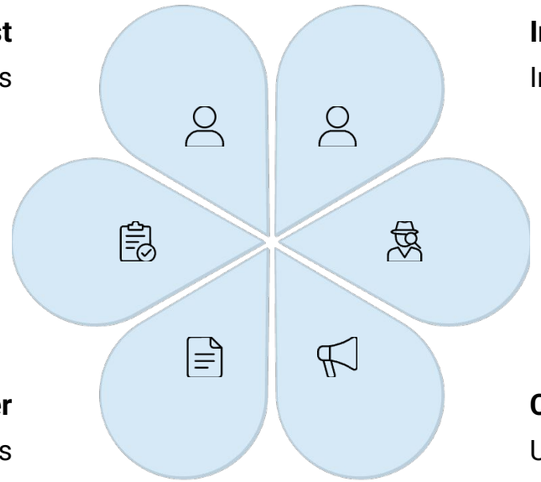
{ik} INTERVIEW KICKSTART

Can Agents Help?

The Tool Overload:

1. Security monitoring dashboard
2. Threat detection software
3. Incident logging system
4. Vulnerability scanners
5. Customer communication platforms
6. Report generation tools
7. Compliance tracking systems

The Role Chaos:



Security Analyst
Monitors threats

Incident Responder
Investigates alerts

Compliance Officer
Ensures regulatory standards

Forensic Specialist
Analyzes attack patterns

Report Writer
Documents incidents

Communications Lead
Updates clients

Source: https://www.microsoft.com/en-us/microsoft-cloud/blog/2025/12/04/multi-agentic-ai-unlocking-the-next-wave-of-business-transformation/

{ik} INTERVIEW KICKSTART

Multi-agent systems to the rescue!

60x

Faster Response Time

From 30 minutes to 30 seconds

93%

Cost Reduction

From $15 to less than $1 per incident

90%

Incident Investigation Automation

Of investigations automated

∞

Scalability

MSPs can scale services without scaling teams

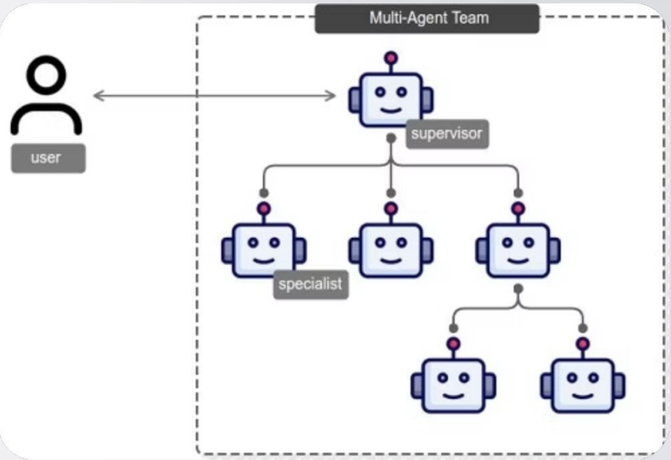
Source: https://www.microsoft.com/en-us/microsoft-cloud/blog/2025/12/04/multi-agentic-ai-unlocking-the-next-wave-of-business-transformation/

{ik} INTERVIEW KICKSTART

What is Multi-Agent System?

Multi-agent systems coordinate specialized components to tackle complex workflows.

However, not every complex task requires this approach — a single agent with the right tools and prompt can often achieve similar results.



Why Multi-Agent Systems?

Context Management

Provide specialized knowledge without overwhelming the model's context window.

Distributed Development

Allow different teams to develop and maintain capabilities independently with clear boundaries.

Parallelization

Spawn specialized workers for subtasks and execute them concurrently for faster results.

Scalable

You can add more agents to a MAS without slowing it down. This helps handle more work and larger amounts of data efficiently.

Strong and reliable

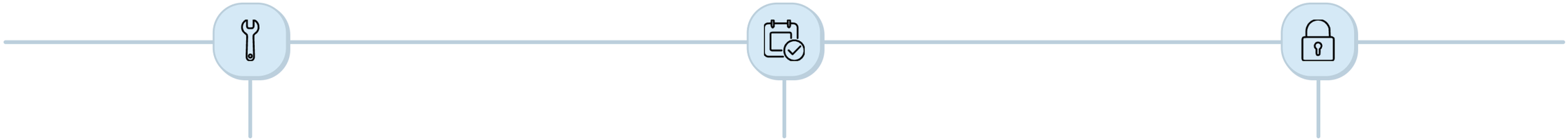
If one agent stops working, the system keeps going because other agents take over. This makes MAS dependable, especially in important situations.

Faster and more efficient

By letting many agents work on different parts of a problem at the same time, MAS can solve problems much quicker and use computer resources better.

{ik} INTERVIEW KICKSTART

When to Build Multi-Agent Systems?



Too Many Tools

When a single agent has too many tools and struggles to make effective decisions on which ones to use.

Specialized Knowledge

For tasks requiring deep, specialized knowledge with extensive context, such as very long prompts or domain-specific tools.

Sequential Constraints

When there's a need to enforce specific sequential constraints, unlocking capabilities only after certain conditions are met.

{ik} INTERVIEW KICKSTART

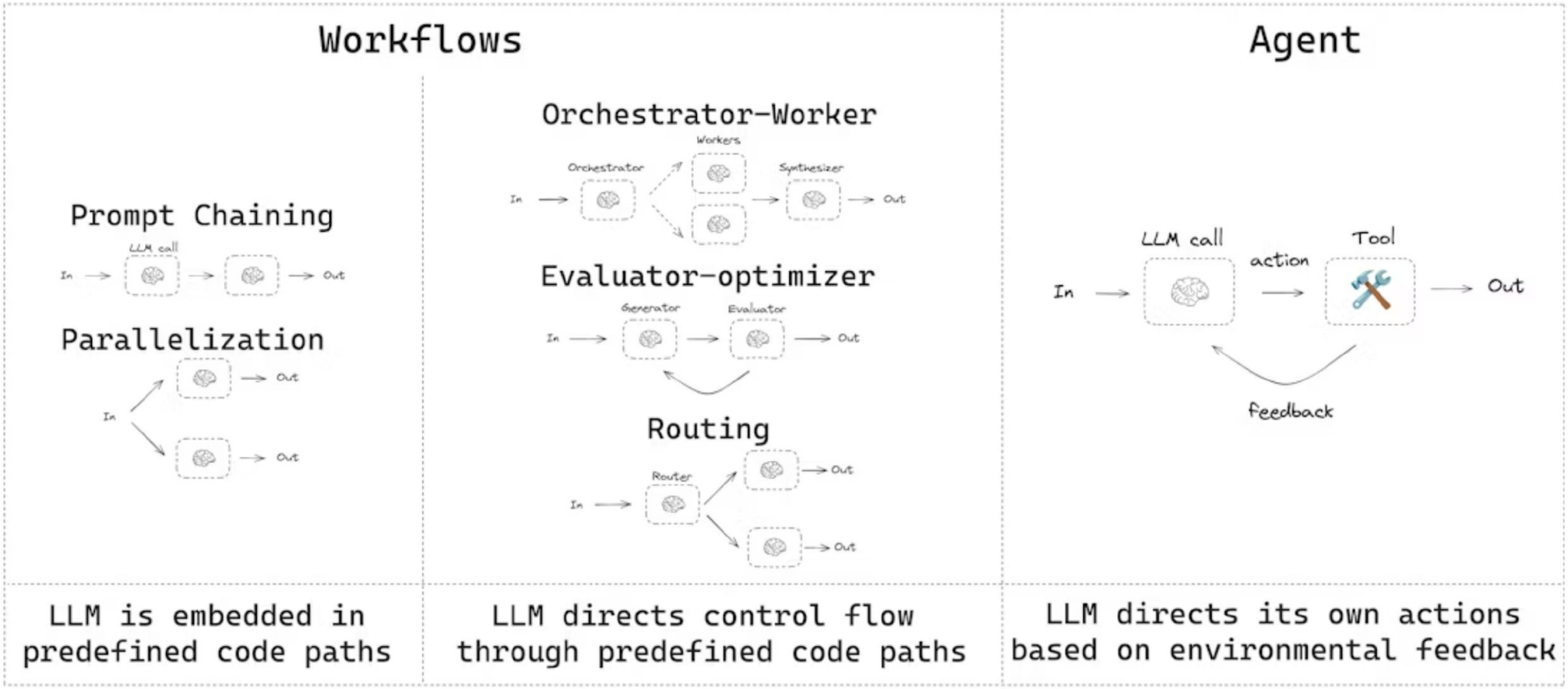


Core Principle

At the center of multi-agent design is **context engineering** — deciding what information each agent sees.

The quality of your system depends on ensuring each agent has access to the right data for its task.

Common Workflows & Agent Patterns



{ik} INTERVIEW KICKSTART

Popular Frameworks to build Multi-Agents

JADE

AutoGen

LangGraph

LangChain

Google ADK

LlamaIndex

CrewAI

...and many more

{ik} INTERVIEW KICKSTART



LangGraph

A low-level orchestration framework and runtime for building, managing, and deploying long-running, stateful agents.

Why LangGraph?

- Fine-grained control over agent flow
- Built-in persistence & human-in-the-loop
- Streaming-first architecture
- Debuggability & observability
- Flexible agent patterns
- Production-ready with LangGraph Platform
- State management done right
- Largest ecosystem & community

{ik} INTERVIEW KICKSTART

LangGraph - Key Concepts

At the heart of LangGraph is the concept of a stateful graph.

1

State

Represents the context or memory that is maintained and updated as the computation progresses.

2

Nodes

Building block of your LangGraph. Each node represents a function or a computation step. Each node performs specific tasks, such as processing input, making decisions, or interacting with external APIs.

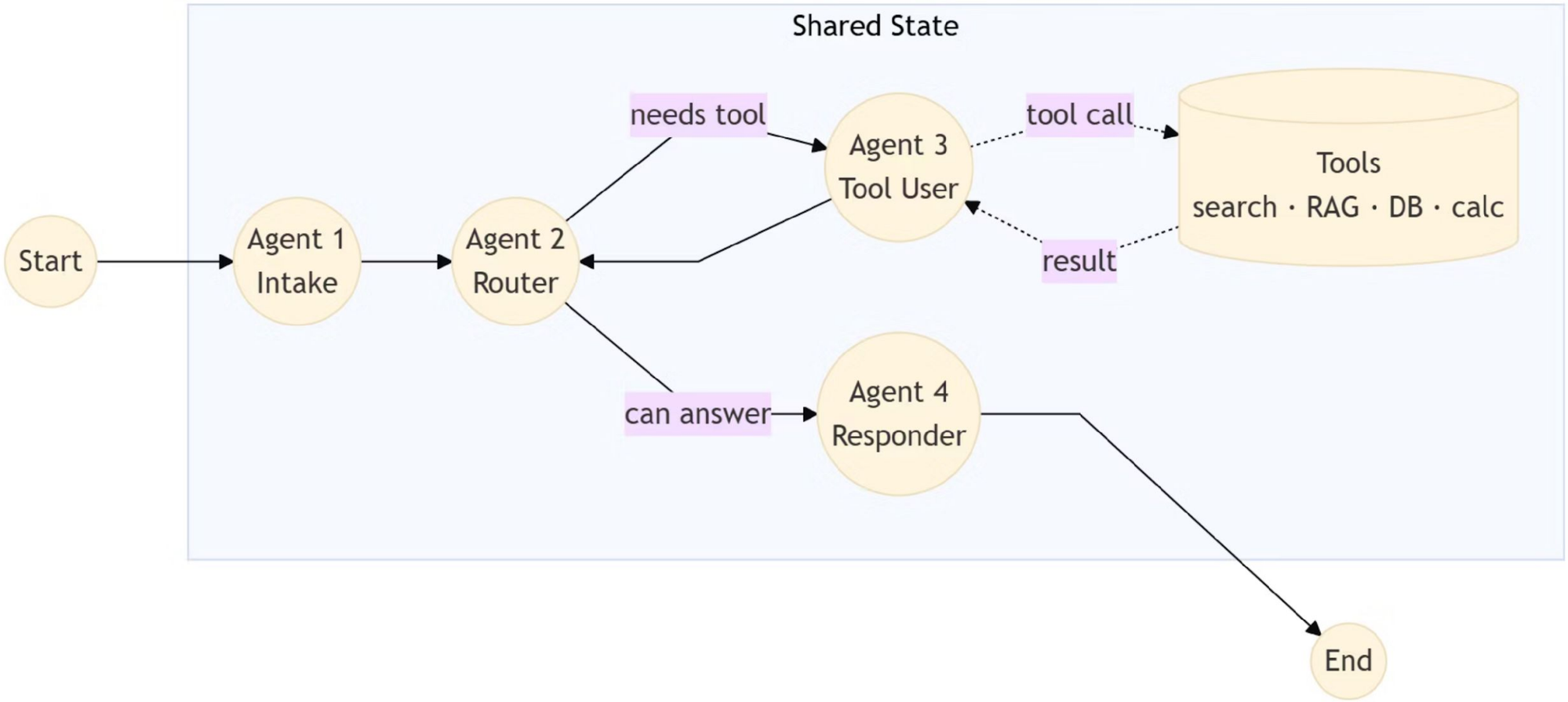
3

Edges

Connect the nodes in your graph, defining the flow of computation. LangGraph supports conditional edges, allowing you to dynamically determine the next node to execute based on the current state of the graph.

{ik} INTERVIEW KICKSTART

LangGraph - Key Concepts



{ik} INTERVIEW KICKSTART

LangGraph - State

```python
from langchain.messages import AnyMessage
from typing_extensions import TypedDict

class State(TypedDict):
    messages: list[AnyMessage]
    extra_field: int
```

State Definition

State use ***reducers*** to control how state updates are processed.

Update State

Our node is just a Python function that reads our graph's state and makes updates to it. The first argument to this function will always be the state.

We will discuss these more in detail while doing code demo

```python
from langchain.messages import AIMessage

def node(state: State):
    messages = state["messages"]
    new_message = AIMessage("Hello!")
    return {"messages": messages + [new_message], "extra_field": 10}
```

{ik} INTERVIEW KICKSTART

Persistence

What are Checkpointers?

LangGraph leverages built-in checkpointers to save the entire state of your agent workflow at specific points, ensuring that no progress is lost and providing a robust foundation for operations.

Why Persistence Matters

Persistence is fundamental for advanced agentic workflows, enabling:

Resuming State: Agents can seamlessly pick up exactly where they left off, even after interruptions or system restarts.

Human-in-the-Loop: Facilitates human intervention for review or decision-making, allowing the agent to continue from that exact point.

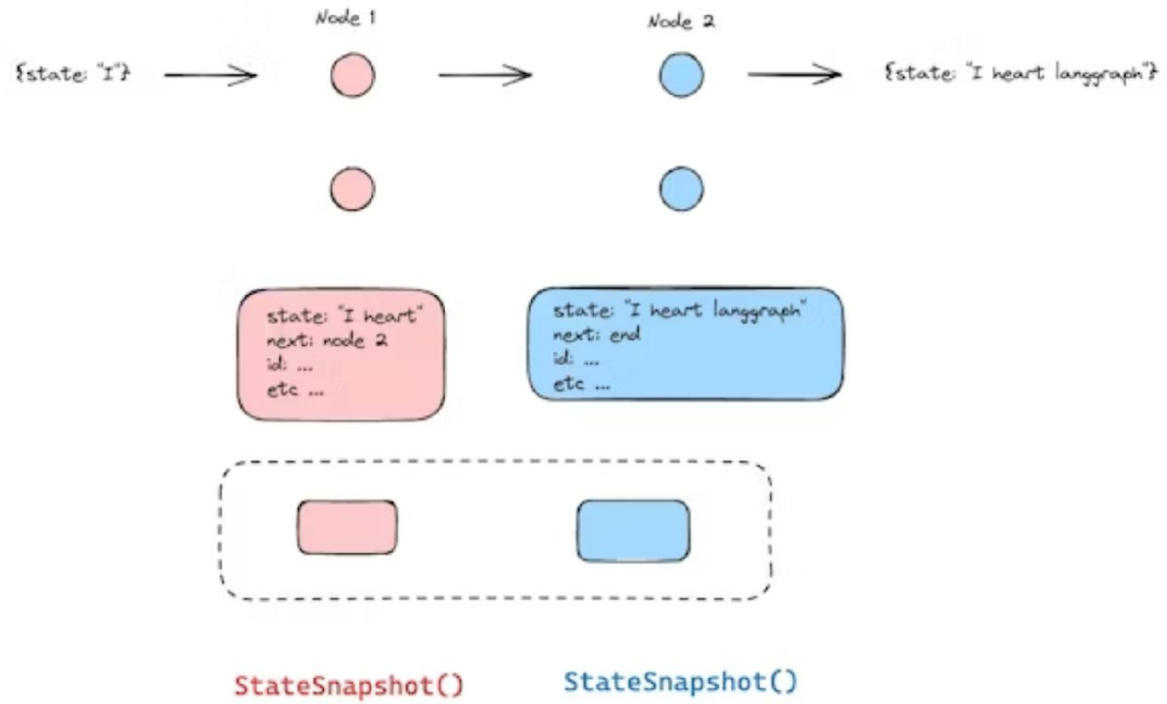
Fault Tolerance: Ensures workflows can recover gracefully from errors or unexpected shutdowns without losing valuable progress or context.

How It Works at a High Level

Checkpointers automatically record the graph's internal state—including messages, tool calls, and variable values—to a persistent store. This mechanism allows the graph to be rehydrated and resume execution from any previously saved checkpoint, ensuring continuity and reliability.

{ik} INTERVIEW KICKSTART

Persistence



Control flow of nodes, edges

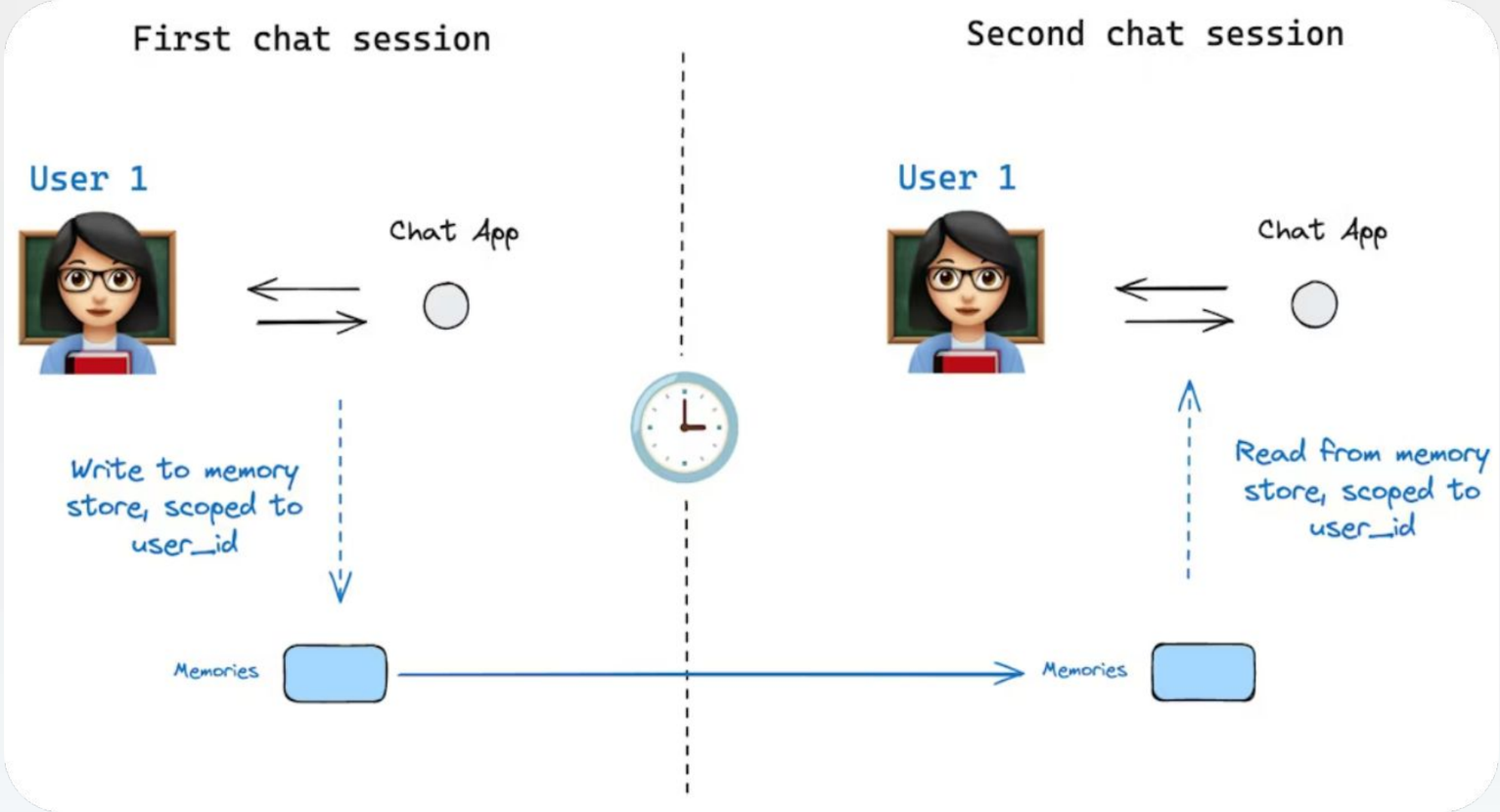
Each sequential node is a separate super-step, while parallel nodes share the same super-step

State and relevant metadata packaged at every super-step

Collection of checkpoints

Type for checkpoints

{ik} INTERVIEW KICKSTART



Memory Store

With checkpointers alone, we cannot share information across threads. This motivates the need for the Store interface. As an illustration, we can define an **InMemoryStore** to store information about a user across threads.

A Simple Graph - Define States & Nodes

```python
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langchain_core.runnables import RunnableConfig
from typing import Annotated
from typing_extensions import TypedDict
from operator import add

class State(TypedDict):
    foo: str
    bar: Annotated[list[str], add]

def node_a(state: State):
    return {"foo": "a", "bar": ["a"]}

def node_b(state: State):
    return {"foo": "b", "bar": ["b"]}
```

{ik} INTERVIEW KICKSTART

A Simple Graph - Build Graph & Compile

```
workflow = StateGraph(State)
workflow.add_node(node_a)
workflow.add_node(node_b)
workflow.add_edge(START, "node_a")
workflow.add_edge("node_a", "node_b")
workflow.add_edge("node_b", END)

checkpointer = InMemorySaver()
graph = workflow.compile(checkpointer=checkpointer)

config: RunnableConfig = {"configurable": {"thread_id": "1"}}
graph.invoke({"foo": "", "bar":[]}, config)
```

{ik} INTERVIEW KICKSTART

Break Time!

Take a 10-minute pause to recharge.



Enjoy a Warm Drink

Sip on your favorite beverage.



Set Your Timer

You'll be refreshed in 10 minutes.



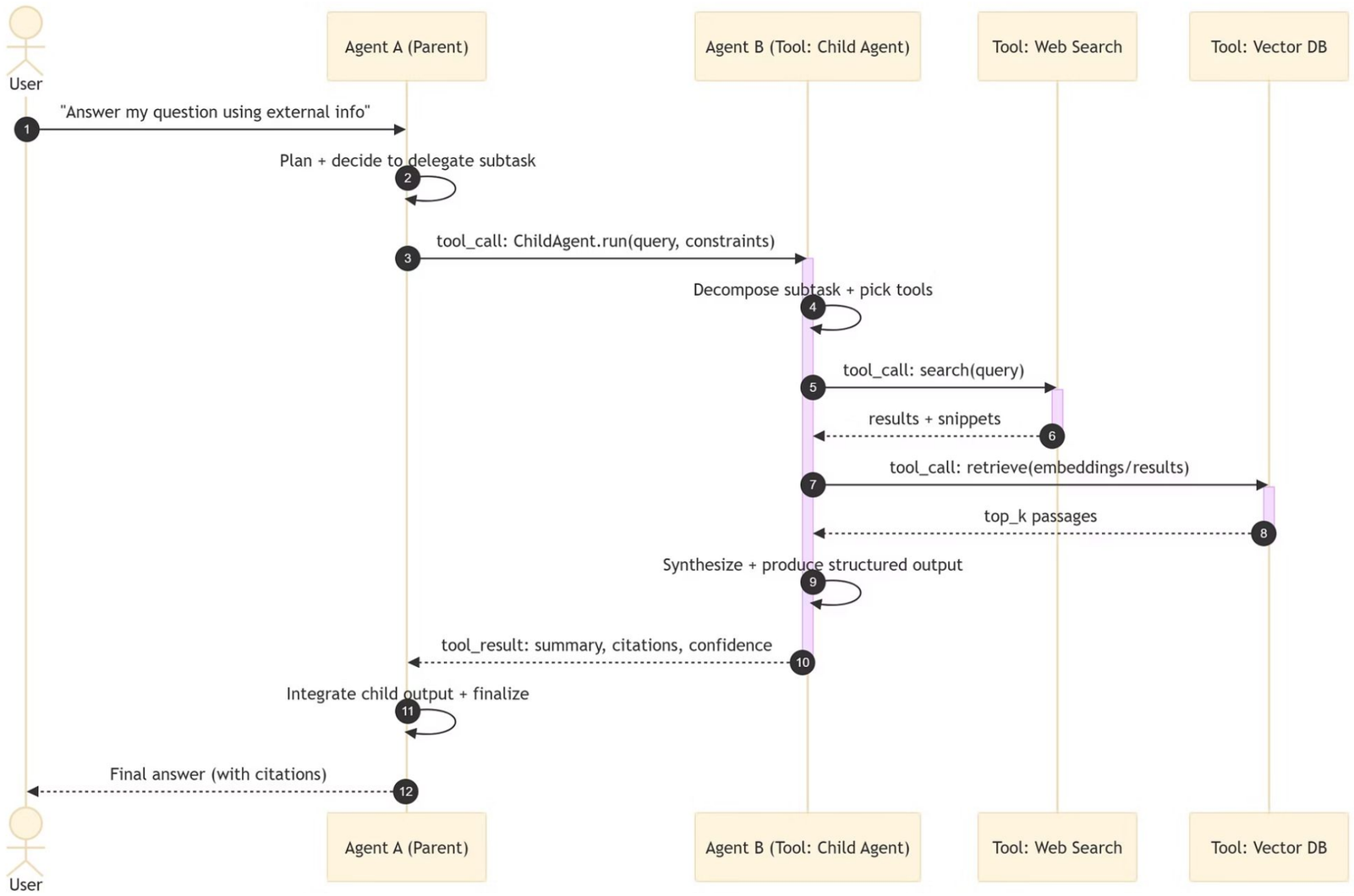
Find Your Zen

Breathe and clear your mind.

{ik} INTERVIEW KICKSTART

SubAgents

Subagents are useful for context quarantine (keeping the main agent's context clean) and for providing specialized instructions.



{ik} INTERVIEW KICKSTART

SubAgents

Subagents isolate this detailed work—the main agent receives only the final result, not the dozens of tool calls that produced it.

When to use subagents:

- Multi-step tasks that would clutter the main agent's context
- Specialized domains that need custom instructions or tools
- Tasks requiring different model capabilities
- When you want to keep the main agent focused on high-level coordination

When NOT to use subagents:

- Simple, single-step tasks
- When you need to maintain intermediate context
- When the overhead outweighs benefits

{ik} INTERVIEW KICKSTART

Let's Get Your Hands Dirty with Code

Time to build! We'll walk through creating a multi-agent system step by step.



{ik} INTERVIEW KICKSTART



What we're building?

A smart Travel Planning Multi-Agent system

An intelligent system that coordinates multiple specialized agents to search flights, hotels, plan itineraries, and provide personalized travel recommendations.

{ik} INTERVIEW KICKSTART

From Idea to Agents:

Step 1 - Start with the process to automate

What the Agent Should Do?

1. Search hotels and flights based on user preferences
2. Plan complete travel itineraries
3. Answer travel-related questions with deep research
4. Handle multiple queries in parallel
5. Present unified, well-formatted travel recommendation

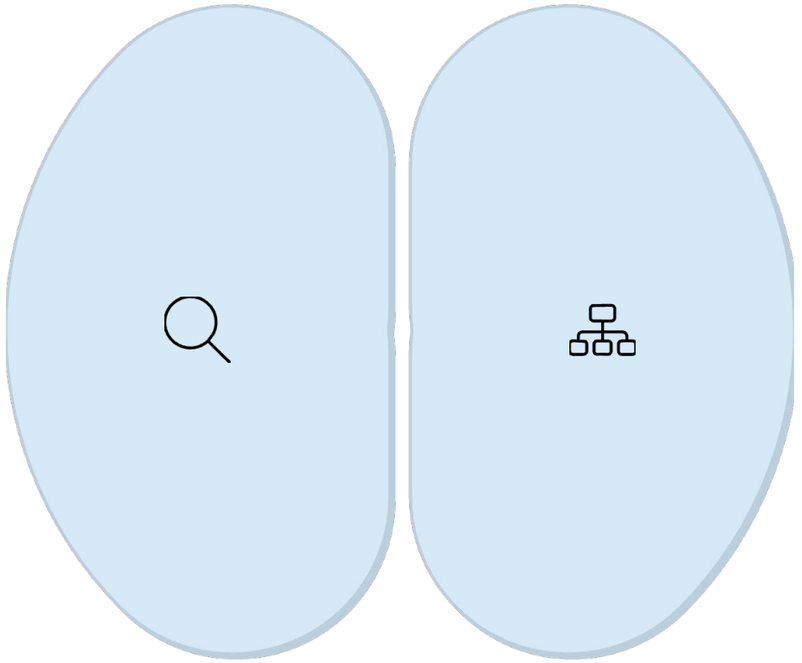


From Idea to Agents:

Step 2 - Map out your workflow as discrete steps

Search Agent

Handles hotel and flight search based on user preferences.



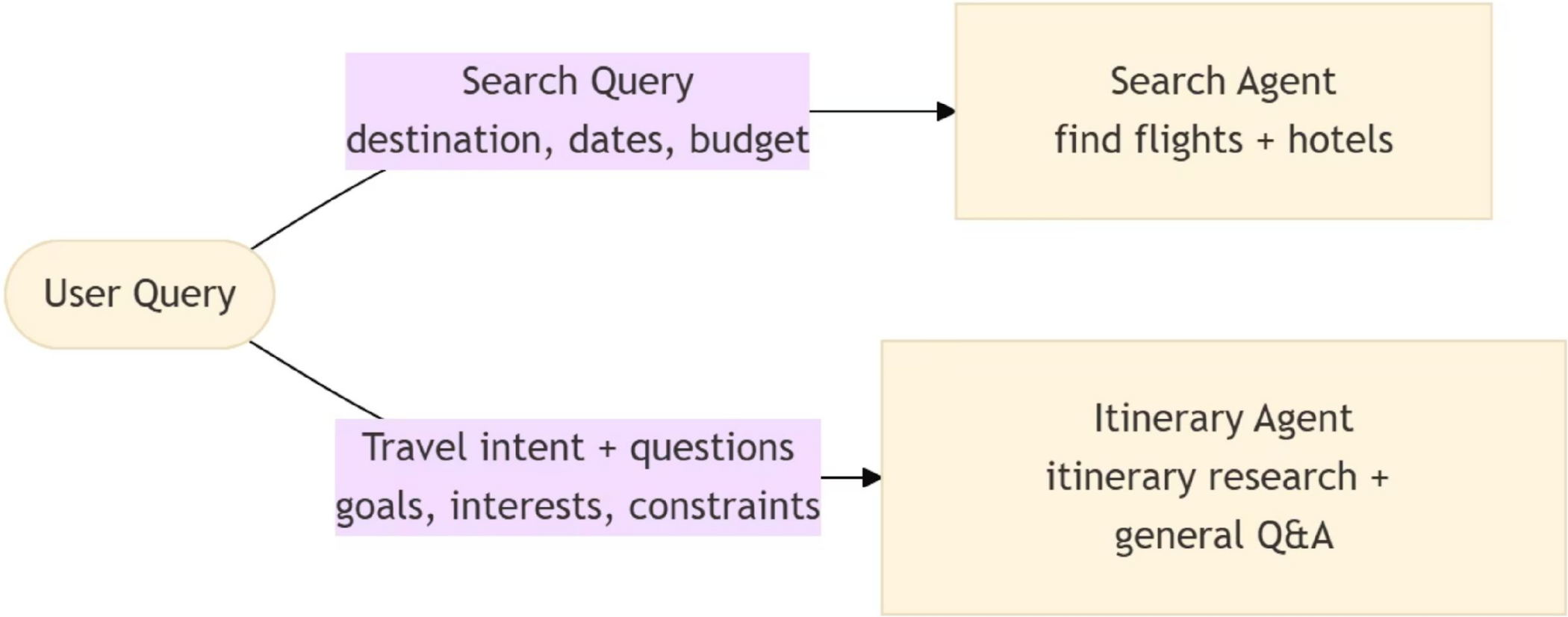
Itinerary Planner Agent

Utilizes search results and research to create detailed travel plans.

{ik} INTERVIEW KICKSTART

From Idea to Agents:

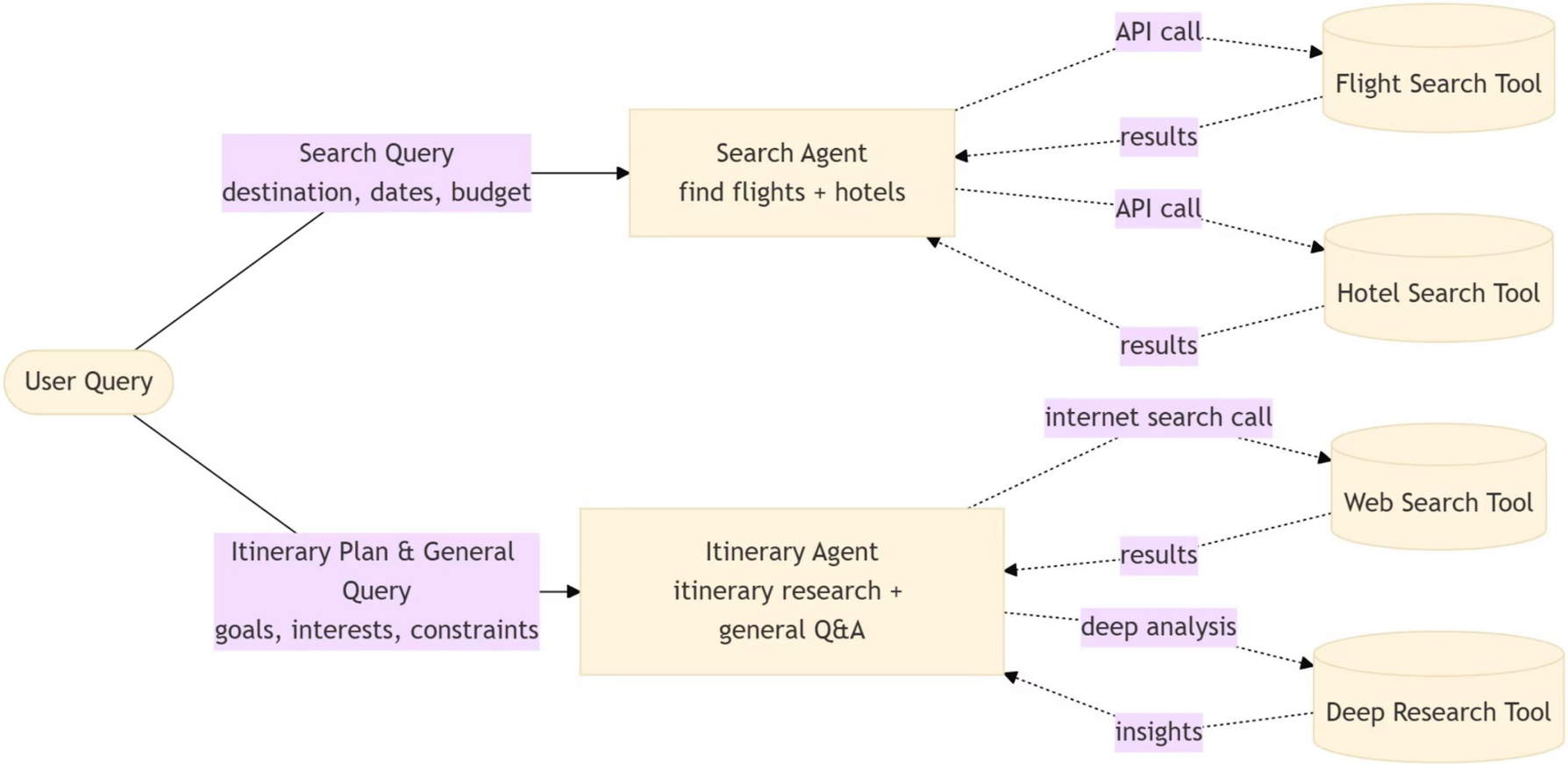
Step 2 - Map out your workflow as discrete steps



{ik} INTERVIEW KICKSTART

From Idea to Agents:

Step 3 - Identify what each steps need to do



{ik} INTERVIEW KICKSTART

API Keys for Search



Hotel & Flight Search

Serp API



Internet Search

Tavily API

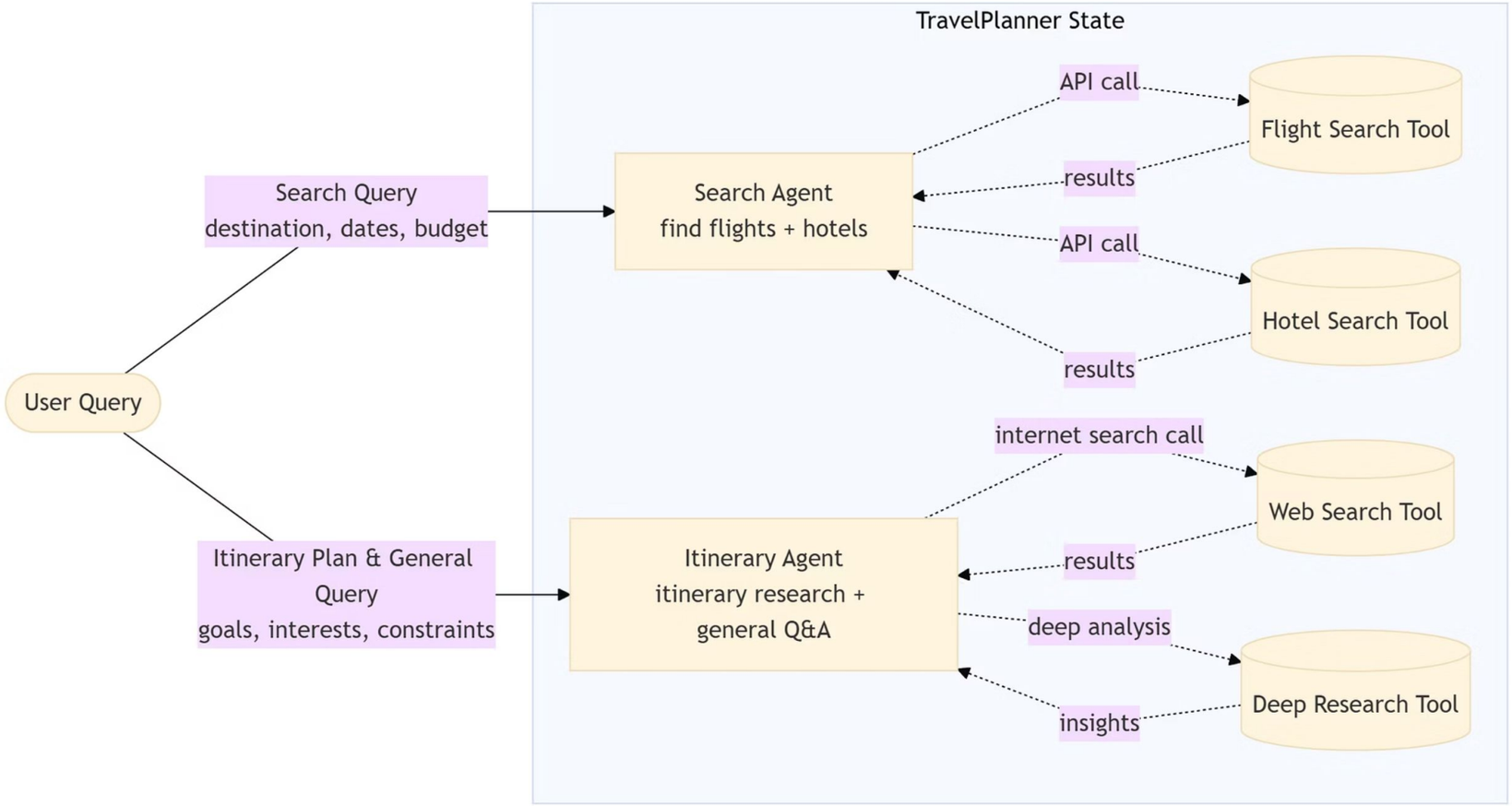
Guide to create API Keys:

https://docs.google.com/document/d/166LFb-7zTmEcG-pXvfsEpY8AunkHJfPXUrvWnlERAcg/edit?usp=sharing

{ik} INTERVIEW KICKSTART

From Idea to Agents:

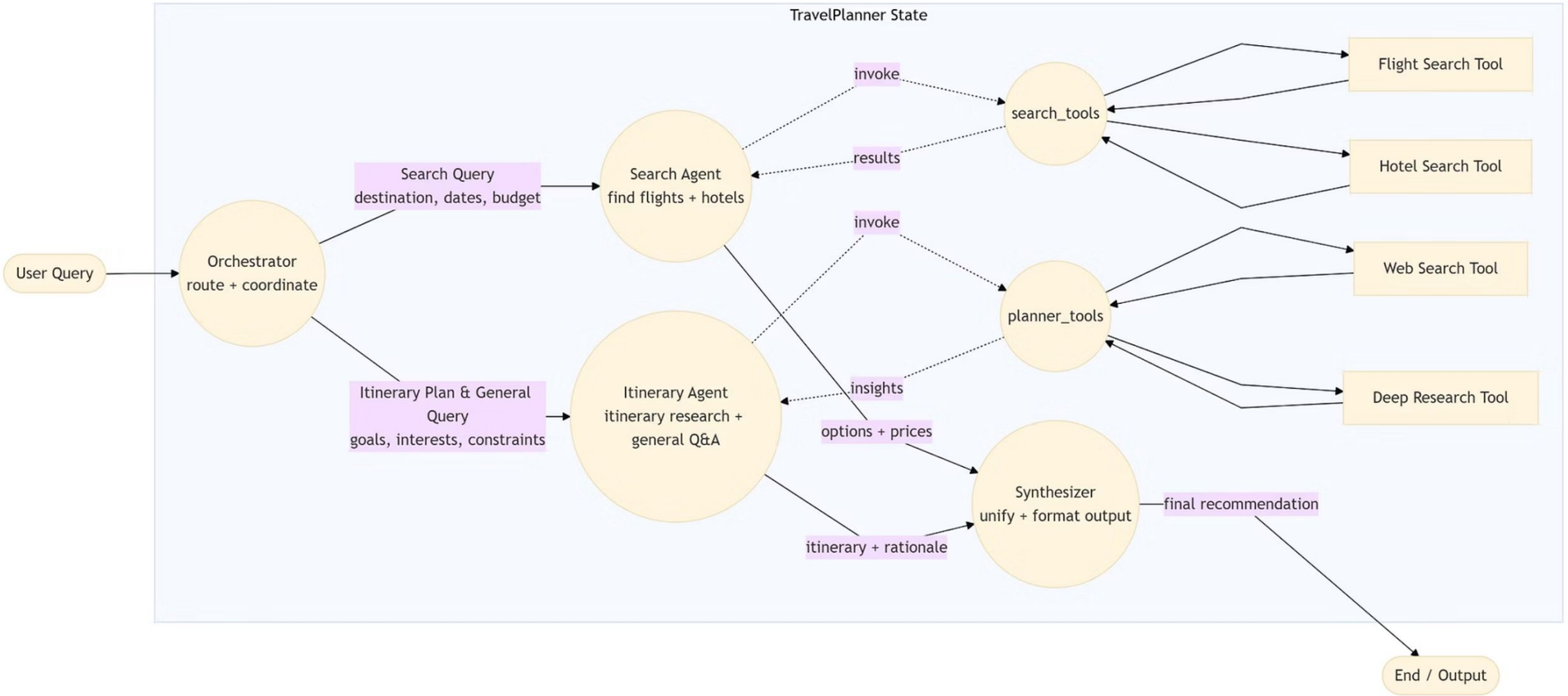
Step 4 - Design Your Agent State



{ik} INTERVIEW KICKSTART

From Idea to Agents:

Step 5 - Build Your Nodes



{ik} INTERVIEW KICKSTART

From Idea to Agents:

Step 6 - Compile Your Workflow & Test the Multi-Agent System

{ik} INTERVIEW KICKSTART





♡ APPRECIATION ✨ GRATITUDE

Thank You!

Questions? Let's Discuss!

{ik} INTERVIEW KICKSTART